Inside the MaskSim Repository

Implementation, data provenance and training procedure - read from the source in C:\Users\aakar\masksim

1. What you currently have
2. The architecture as actually implemented
3. Where the data actually comes from
4. The exact training configuration
5. How long training took
6. Step-by-step: training this yourself
7. Practical cautions
8. Three things in this code worth taking into your own implementation

A file-by-file reading of the official MaskSim repository as it sits on your machine, with the exact data provenance, the exact training configuration, and the steps to train it yourself.

Everything below was read from the source in your folder, not from the paper. Where the code and the paper disagree, the code wins and I say so.

---

1. What you currently have

Item	State
Repository source (`train.py`, `src/`, `preprocess.py`, `evaluate.py`)	✅ present
`checkpoints/JPEG_Qrandom_w512/newsynth/*.ckpt`	✅ 7 detectors, 23.2 MB each
`checkpoints.zip`	✅ 139 MB (can delete — already extracted)
`calibration/*.csv`	✅ 14 files, the FPR lookup tables
`src/third_party/.../DenoiserWeight/model_best.th`	✅ 2.3 MB — the pretrained DnCNN
`data/` and `processed_data/`	**❌ absent — you have no training data**
Your run outputs (`sim_map_*.html`, `wolf_result.txt`)	✅ 7 maps + the score log

This is the critical gap. You downloaded `checkpoints.zip` but never ran `data_preparation.sh`. You can *detect* with the authors' weights; you cannot *train* until the data is on disk.

---

2. The architecture as actually implemented

The paper's Figure 2 is accurate but omits three things that are in `src/masksim.py`. Two of them matter a lot.

2.1 The mask is spatially smoothed — and this is not in the paper

`src/masksim.py:168-173`:

```
self.mask_conv = nn.Conv2d(in_channels=self.channels, out_channels=self.channels,
                           groups=self.channels, kernel_size=kernel_sz, bias=False,
                           padding=(kernel_sz - 1) // 2)
weights = torch.ones((kernel_sz, kernel_sz))
weights = weights.view(1,1,kernel_sz,kernel_sz).repeat(self.channels,1,1,1) / (kernel_sz ** 2)
self.mask_conv.weight = nn.Parameter(weights)
self.mask_conv.requires_grad_(False)          # <-- FIXED, never trained
```

and `get_mask()` at line 239:

```
mask = self.mask_conv(mask_pre)   # fixed 3x3 box blur
mask = torch.sigmoid(mask)
```

So the mask is **not** `sigmoid(parameter)`. It is:

> **`M = sigmoid( box_blur_3×3( mask_pre_activation ) )`**

A fixed, non-trainable averaging filter sits between the free parameters and the sigmoid. Because the blur is in the computational graph, gradients are spread across each 3×3 neighbourhood, so neighbouring frequency bins cannot move independently. **This is a built-in spatial-smoothness regulariser on the mask**, and it is the single most important implementation detail that neither the paper nor the README mentions.

This directly explains something from my own clean-room implementation: my trained masks came out visibly noisy and underperformed the closed-form baseline. I had `sigmoid(parameter)` with no smoothing. Real artifacts are spatially clustered in frequency; the box blur encodes that prior for free. **If you write your own version, put this in.**

2.2 The DnCNN changes the spatial size: 512 → 478

`src/masksim.py:147-153`, with the authors' own comment:

```
# For DnCNN, the output size is 478
if preprocess == "DnCNN":
    out_dncnn = self.preproc(torch.randn(1, channels, img_size, img_size))
    ref_pattern = torch.randn(out_dncnn.shape[1:4]) * 0.01
```

The denoiser uses valid padding, so a 512×512 crop emerges as 478×478, and the mask and reference are allocated at **478×478, not 512×512**. That is exactly the shape of the heatmap inside your `sim_map_sdxl.html` — mystery solved.

Consequence: the mask and reference are 3 × 478 × 478 = **685,926 parameters each**, which is where the paper's "0.7M for the mask and 0.7M for the reference" comes from.

2.3 Normalisation is BatchNorm1d on the flattened vector

`src/masksim.py:181` — `self.norm_vector = nn.BatchNorm1d(self.channels, affine=True)`, applied after `ffts_masked.view(B, C, -1)`. Not a BatchNorm2d over the spectrum. A small difference, but it changes what "per-channel statistics" means.

2.4 What the paper does get exactly right

`training_step`, line 369, with the authors' own comment:

```
# original: abs negative similarity
logits_neg = torch.exp(self.clf_a) * similarity_neg.abs() + self.clf_b
```

The `|cos|` on pristine images is there, exactly as Eq. 7 specifies, and `validation_step` uses the unsigned similarity. The training/inference asymmetry is real and deliberate.

2.5 A parameter the paper never mentions

`MaskSim.__init__` takes `num_masks` (default 1) and builds `ParameterList`s of masks and references. In `train_on_multi_classes` there is a commented-out `#num_masks=4`. So the authors built multi-template support, tried it for the multi-class case, and shipped with `num_masks=1`. Worth knowing if you pursue the multi-reference idea — the scaffolding already exists.

---

3. Where the data actually comes from

Read from `download.py`, `data_preparation.sh`, `train.py` and `preprocess.py`. There are **four** distinct sets, and confusing them is easy.

3.1 Training — pristine (real)

`train.py:98-101`

```
img_dir_real_list = ["processed_data/train/mit5k",
                     "processed_data/train/coco_train",
                     "processed_data/train/dresden"]
```

- **MIT-Adobe FiveK** — 5000 processed photos, TIFF
- **COCO train split** — 5000 JPEGs
- **Dresden Image Database** — 1488 camera-forensics images, TIFF

Source: `train.zip` from `cirrus.universite-paris-saclay.fr/s/2eabgG8fZy8nXME`, or the bundled `data.tar.gz`.

3.2 Validation — pristine

`train.py:103-104`

```
real_valid_folder_list = ["processed_data/train/hdrburst",
                          "processed_data/train/coco_val"]
```

- **HDR+ Burst** — 153 raw images
- **COCO val split** — the other half of COCO

3.3 Training + validation — synthetic (fake)

`train.py:106` → `data/newsynth/{class}`, and `TRAINING_SET = "newsynth"`.

This is **not** Synthbuster. `newsynth` is built by `download.py:download_newsynth()`, which reads `newsynth.csv` and downloads each image individually from a URL, one column per generator:

CSV column	folder
`URL Stable Diffusion 1.*`	`sd1`
`URL Stable Diffusion 2.1`	`sd2`
`URL Stable Diffusion XL`	`sdxl`
`URL Dall.e 2`	`dalle2`
`URL Dall.e 3`	`dalle3`
`URL midjourney`	`midjourney`
`URL firefly`	`firefly`

The train/val split of the fakes is random, not by source (`train.py:156`):

```
fake_train_dataset, fake_valid_dataset = torch.utils.data.random_split(fake_dataset, [0.9, 0.1])
```

90% train, 10% validation, reshuffled every run — so the validation fakes differ between runs unless you seed it.

3.4 Test — a completely separate set

`preprocess.py:13-19` (fake) and `preprocess.py:157` (real):

```
"data/synthbuster/stable-diffusion-1-3", ... "data/synthbuster/firefly"
in_folder = "data/raise"
```

- **Fake test: Synthbuster**, from Zenodo `10.5281/zenodo.10066460`
- **Real test: RAISE**

This is the design point that makes the evaluation trustworthy. **Synthbuster's prompts were written to match the RAISE photographs**, so test-time reals and fakes are semantically paired — content cannot act as a shortcut. And the test fakes come from a *different collection* than the training fakes, so there is no leakage between them.

3.5 Calibration — the source of those FPR percentages

The 14 CSVs in `calibration/` are precomputed detector scores on known-real images:

- `coco_unlabled_2017_to_{class}.csv` — COCO unlabeled 2017
- `FlickrFace_to_{class}.csv` — FlickrFaces

When your wolf run printed "*FPR corresponding to the score: 32.8%*", it binary-searched your score into the sorted list from one of these files. **That is the number those files exist to produce**, and it is why the tool reports an error rate rather than thresholding at 0.5.

3.6 Summary

Split	Real	Fake
Train	MIT-5k + COCO-train + Dresden	`newsynth` (90%)
Validation	HDR-Burst + COCO-val	`newsynth` (10%)
Test	**RAISE**	**Synthbuster**
Calibration	COCO-unlabeled-2017, FlickrFaces	—

---

4. The exact training configuration

From `train.py:50-57` and `masksim.py:543-556`:

Setting	Value	Source
Image size	512 × 512 (→ 478 after DnCNN)	`IMG_SIZE = 512`
Batch size	8	`BATCH_SZ = 8`
Max epochs	50	`MAX_EPOCHS = 50`
Learning rate	1e-3 all parameters	`LEARNING_RATE = 1e-3`
LR for DnCNN	**1e-4**	`configure_optimizers`
Optimiser	Adam, `weight_decay=0.0`	`configure_optimizers`
Scheduler	`ExponentialLR(gamma=0.99)`	`configure_optimizers`
Preprocessing	DnCNN, **fine-tuned** (`preproc_freeze=False`)	`train.py:108`
Colour space	YCbCr when compressing, else RGB	`train.py:83`
Dataloader workers	8	`NUM_WORKERS = 8`
Accelerator	**`gpu`, `devices=[0]` — one GPU**	`train.py:193`
Masks per model	1	`num_masks=1`
Checkpoint kept	**minimum validation loss**	only `low_loss_callback` is passed

Augmentation

JPEG, `dataset.py:33-41` — applied with p=0.95, quality uniform in **[65, 100]**, to *both* classes.

Resizing — applied to pristine images ONLY. `dataset.py:203-215`:

```
if self.mode == "train":
    if label == 0:                                  # <-- REAL IMAGES ONLY
        if np.random.rand() < 0.5:
            up_ratio = self.img_size / min(img.shape[0], img.shape[1]) * np.random.uniform(1,
1.5)
            img = skimage.transform.resize(img, (final_h, final_w), anti_aliasing=False)
    img = random_crop(img, self.img_size)
```

Half of all real training images are rescaled by a random factor; **synthetic images are never rescaled**.

The intent is clear — pristine images are enormous camera files and must be brought to a comparable scale, and the resize also destroys native sensor high-frequency structure that would otherwise be a shortcut. But it has a consequence worth stating plainly:

> **During training, "shows resampling traces" correlates with "real".** The model is taught, implicitly, that a rescaled image is pristine.

That is a concrete mechanism for the paper's own §5 limitation — that rescaled synthetic images evade detection. It is not merely that the mask looks at fixed frequency bins; the training signal actively pushes resampled images toward the "real" class. If you want to attack that limitation, this asymmetry is the first thing to change: apply identical resize augmentation to both classes, or to neither.

Class balance: `balanced_concat_dataloader` (`dataset.py:116`) uses a `WeightedRandomSampler` with weights `1/len(dataset)` per set, so each batch draws real and fake equally regardless of the size imbalance.

---

5. How long training took

Your checkpoint filenames record the *epoch selected*, not the wall-clock:

detector	best epoch (of 50)	valid AUROC	valid loss
`sdxl`	37	1.00	0.033
`dalle2`	37	1.00	0.065
`firefly`	44	1.00	0.035
`sd1`	44	1.00	0.041
`sd2`	44	1.00	0.063
`dalle3`	47	1.00	0.036
`midjourney`	48	1.00	0.102

All seven reach **validation AUROC 1.00** — on *validation* fakes drawn from the same `newsynth` collection as training. That is not the test number; the honest figures are in the paper's Table 2 (SD-2 detector: 99.1 on SD-2, 68.2 on DALL·E 2).

Every file carries mtime `1717657402000`–`1717657404000` — all seven within **two seconds** of each other, which is when the archive was packaged (June 2024), not when training ran. **The wall-clock is not recoverable from your folder.**

The only authoritative figure is the paper, §4.5:

> *"The training time of one detector using 2 NVIDIA A100 GPUs and 8 CPUs at 3.1 GHz was 3 hours."*

So: **~3 hours per detector, ~21 GPU-hours for all seven**, at 50 epochs, batch 8, 512×512, with DnCNN fine-tuning.

Two adjustments for your situation:

- **The released `train.py` uses a single GPU** (`devices=[0]`), not the paper's two. Expect meaningfully longer than 3 hours per detector on one A100.
- **On a consumer GPU** (RTX 3060/4070 class), budget 8–15 hours per detector. On **CPU only** — the `accelerator="cpu"` line is commented out at `train.py:194`, so it is supported — fine-tuning a DnCNN at 512² for 50 epochs is impractical; days per detector.

If you have no GPU, the realistic paths are: set `PREPROC = None` (drop DnCNN, which the paper's §5 says costs little), drop `IMG_SIZE` to 256, and reduce epochs — which is the configuration in the project I built for you.

---

6. Step-by-step: training this yourself

Step 0 — you need the data first (the long pole)

`data_preparation.sh` is a bash script; on Windows run the steps by hand in PowerShell:

```
mkdir cache, data, processed_data
curl.exe -L -o cache\data.tar.gz
"https://avocat.ovh.hw.ipol.im/static/yanhao/masksim/data.tar.gz"
tar -xzf cache\data.tar.gz
```

This single bundle contains `coco_train`, `coco_val`, `dresden`, `hdrburst`, `mit5k`, `newsynth` — everything for training and validation. Use `curl.exe`, not `Invoke-WebRequest`, which buffers multi-GB files in RAM.

If that URL is dead, fall back to `download.py`: uncomment `download_pristine()` and `download_synthbuster()` in `__main__`, and note that `download_newsynth()` needs `newsynth.csv`, which is not in your folder — you would have to get it from the repo or the authors.

Step 1 — build the folder layout

`data_preparation.sh` uses symlinks; Windows equivalents:

```
cd processed_data
mkdir train
cmd /c mklink /D train\mit5k       ..\..\data\mit5k
cmd /c mklink /D train\coco_train  ..\..\data\coco_train
cmd /c mklink /D train\coco_val    ..\..\data\coco_val
cmd /c mklink /D train\dresden     ..\..\data\dresden
```

```
cmd /c mklink /D train\hdrburst    ..\..\data\hdrburst
cmd /c mklink /D train\newsynth    ..\..\data\newsynth
```

(Needs an elevated prompt or Developer Mode. Copying the folders works too, at the cost of disk.)

Step 2 — preprocess the test set

```
python preprocess.py
```

Centre-crops Synthbuster and RAISE to 512 and writes JPEG-compressed variants into `processed_data/JPEG_Qrandom/`. Required before `evaluate.py`; not required for training.

Step 3 — train one detector

```
python train.py -w 512 -b 8 -e 50 -p DnCNN -Q random --compression jpeg --progress
```

Loops over all seven classes in `TRAIN_CLASS_NAMES`. **Edit that list in `train.py:42` to train just one** while you verify the pipeline — do not start a 21-GPU-hour run before you know it works.

Checkpoints land in `checkpoints/JPEG_Qrandom_w512/newsynth/` and will collide with the authors' files, which are named identically. **Back those up first**, or set `VERSION` so it writes to `checkpoints_<VERSION>/`.

Step 4 — evaluate

```
python evaluate.py -Q random -w 512 --compression jpeg --img_q random
```

Step 5 — compare against the authors'

You now have two sets of weights for the same generator. Plot your mask beside theirs — agreement validates your run; disagreement is a finding worth chasing. This is the single best figure you can produce from this repository.

---

7. Practical cautions

Back up `checkpoints/` before training. Same filename pattern, no warning, and re-downloading is 139 MB.

`kaleido==0.2.1` hangs on Windows. You already hit this. `detect_one_image.py` is patched to `write_html`; `evaluate.py` may still call `write_image` — check before a long run.

Python 3.10 or 3.11 only. `numpy==1.23.5` and `torch==2.0.1` have no Windows wheels for 3.12+. Your `.venv` is 3.10 (from the `.pyc` filenames), so you are fine.

Validation fakes are re-sampled every run (`random_split` with no seed), so two runs are not strictly comparable. Add a generator if you need reproducibility.

Validation AUROC 1.00 is not a test result. Validation fakes come from the same `newsynth` collection as training. Only `evaluate.py` against Synthbuster + RAISE gives a number you can report.

---

8. Three things in this code worth taking into your own implementation

1. **The fixed 3×3 box blur on the mask before the sigmoid.** A free spatial-smoothness prior that matches the physics — real artifacts are clustered in frequency. My clean-room version omitted it and produced measurably noisier masks.
2. **Calibration tables instead of a threshold.** Precomputed scores on known-real images, turning an arbitrary number into "*X% of real photographs score this high*". Copy this pattern in anything you deploy.
3. **Test on a different collection from training.** Training fakes come from `newsynth`, test fakes from Synthbuster, and the test reals are prompt-matched to the test fakes. Most detection papers do not separate their fake sources this carefully, and it is why the paper's numbers are believable.

And one thing to change: the **resize-augment-reals-only asymmetry** teaches the model that resampled images are pristine. That is a plausible mechanism behind the method's most serious stated limitation, and fixing it is a genuine, cheap research contribution.